Capa de Aplicación

Resumen para el parcial · Telecomunicaciones

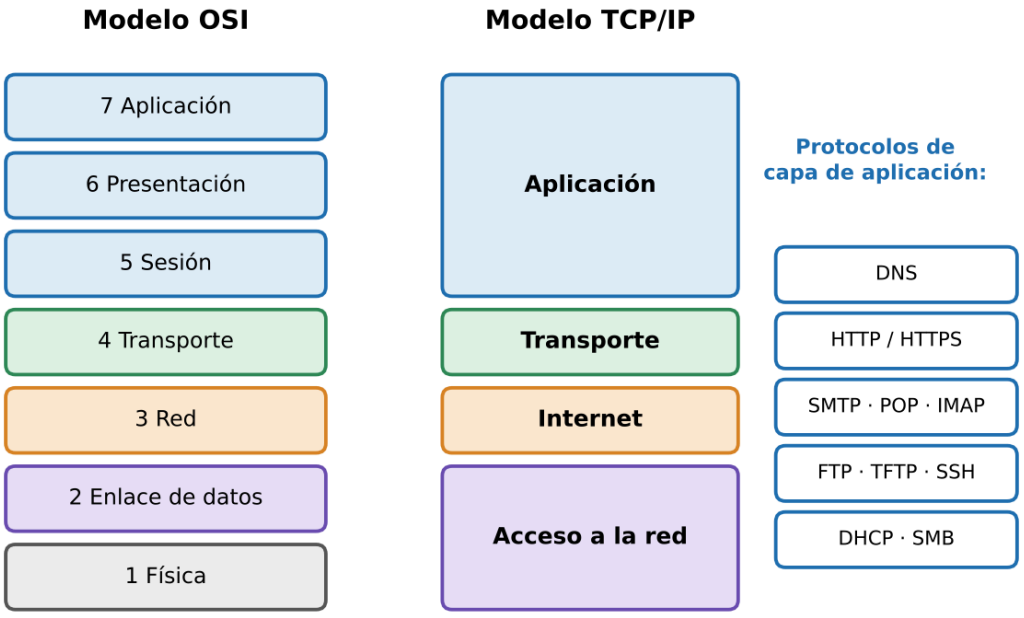
Basado en las diapositivas de la Universidad de Palermo y en el apunte "Capa de Aplicación y Aplicaciones Distribuidas"

Contenido: 1. La capa de aplicación · 2. Puertos · 3. DHCP · 4. DNS · 5. Correo (SMTP, POP, IMAP) · 6. HTTP / HTTPS · 7. FTP y TFTP · 8. SSH · 9. SMB · 10. Tabla final y claves para el parcial

1. La capa de aplicación

Es la **séptima capa y la superior** de los modelos OSI y TCP/IP. Funciona como **interfaz entre las aplicaciones que usa la persona y la red** por la que viajan los mensajes.

- **No es la aplicación en sí**: es el conjunto de **protocolos y servicios** que las aplicaciones usan para intercambiar datos. Chrome o Firefox están "por encima" de la capa y se apoyan en ella.
- TCP/IP se desarrolló **antes** que OSI, y su capa de aplicación equivale a las **3 capas superiores de OSI** (aplicación, presentación y sesión).
- Sus protocolos nacieron antes de las PC, las interfaces gráficas y el multimedia, por eso implementan **muy poco** de lo que OSI define para presentación y sesión.
- Para que la comunicación funcione, los protocolos de **origen y destino deben coincidir** (ser compatibles).
- Una misma aplicación puede usar **varios protocolos a la vez**. Ej.: al abrir una web, el navegador usa **DNS** (resolver el nombre) y **HTTPS** (transferir la página).



Las 3 capas superiores de OSI equivalen a la capa de aplicación de TCP/IP

Figura 1. Modelos OSI y TCP/IP, y protocolos de la capa de aplicación

Aplicaciones, servicios y protocolos

Componente	Qué es	Ejemplos
Aplicación	Programa que usa la gente para comunicarse por la red. Da la forma de crear mensajes.	Navegador web, cliente de correo

Componente	Qué es	Ejemplos
Servicio	Programa que establece la interfaz con la red y prepara los datos para la transferencia. Es **transparente** para el usuario.	Transferencia de archivos, cola de impresión en red
Protocolo	Reglas y formatos que regulan el trato de los datos.	HTTP, SMTP, DNS

Un mismo programa puede usar los tres componentes e incluso el mismo nombre: al decir "**Telnet**" podemos referirnos a la aplicación, al servicio o al protocolo.

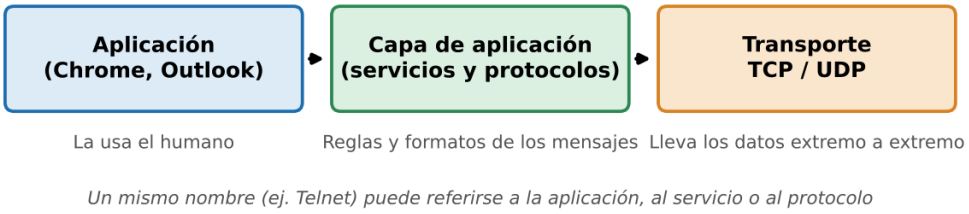


Figura 2. Aplicación, capa de aplicación (servicios y protocolos) y transporte

Funciones de los protocolos de aplicación

- Establecen **reglas consistentes** para intercambiar datos entre aplicaciones y servicios de los dispositivos.
- Definen los **procesos** en cada extremo, los **tipos de mensajes** (solicitudes de servicio, acuses de recibo, datos, estado, error) y su **sintaxis**.
- Definen el **significado de los campos** y **cómo se envían** los mensajes y la respuesta esperada (los **diálogos**).
- Definen la **interacción con la capa inferior**.

2. Puertos de los protocolos principales

Protocolo	Función	Transporte / Puerto
DNS	Traduce nombres de dominio a IP	TCP/UDP 53
HTTP	Web	TCP 80
HTTPS	Web segura (HTTP + TLS)	TCP 443
SMTP	Envío de correo	TCP 25 (587 con cifrado)
POP3	Recepción de correo (descarga)	110 (en las diapositivas figura UDP)
IMAP	Recepción de correo (sincroniza)	143
Telnet	Acceso remoto sin cifrar	TCP 23
DHCP	Configuración automática de red	UDP 67 (servidor) / 68 (cliente)
FTP	Transferencia de archivos	TCP 20 (datos) y 21 (control)
TFTP	Transferencia simple de archivos	UDP 69
SMB	Compartir archivos e impresoras en LAN	445
SSH	Acceso remoto seguro	(no figura en los apuntes; habitualmente TCP 22)

Nota: en la práctica POP3 corre sobre TCP; la diapositiva lo muestra como UDP. Si te lo preguntan, fijate qué indica el material de la cátedra.

3. DHCP (Dynamic Host Configuration Protocol)

Protocolo **cliente/servidor** que permite que un dispositivo obtenga **automáticamente su configuración de red**, sin configurarla a mano. El servidor tiene una lista de IP dinámicas y las asigna a los clientes a medida que quedan libres. Sabe en todo momento **quién tuvo la IP, cuánto tiempo y a quién se la asignó después**.

Entrega: **dirección IP, máscara de subred, puerta de enlace (gateway), servidores DNS y tiempo de concesión (lease)**. Ventaja: automatiza la configuración, evita errores manuales y simplifica la administración en redes grandes.

Métodos de asignación de IP

Método	Cómo funciona
Automática	DHCP asigna una IP **permanente** a un cliente.
Dinámica	Asigna una IP por un **tiempo específico** (o el que pida el cliente). Es el **único** que permite **reusar** direcciones que ya no se necesitan.
Manual	El **administrador** define la IP del cliente y DHCP solo la transmite. Se usa para corregir errores en redes que por alguna razón no usan DHCP.

Tareas de configuración del servidor

- **Ámbito (scope):** cada subred necesita su propio ámbito; un servidor DHCP puede administrar varios.
- **Intervalo de direcciones IP:** IP inicial y final del ámbito (el asistente deduce la máscara de subred).
- **Exclusiones:** IP que **no** se deben asignar (servidor con IP estática, gateway, impresoras de red).
- **Duración de la concesión (lease):** tiempo que el cliente puede usar la IP.
- **Router (puerta de enlace predeterminada):** uno o más routers que deben usar los clientes.
- **Nombre de dominio y servidores DNS** que usarán los clientes para resolver nombres.

Proceso DORA

Es el intercambio de 4 mensajes para asignar una IP. **Todos los mensajes iniciales van en broadcast** (255.255.255.255) porque el cliente aún no tiene IP ni conoce al servidor.

1. **D – Discover:** el cliente (sin IP, origen 0.0.0.0:68) envía un broadcast buscando un servidor DHCP.
2. **O – Offer:** el servidor ofrece una IP libre con máscara, gateway, DNS y lease.
3. **R – Request:** el cliente acepta la oferta (también en broadcast, para avisar a otros servidores DHCP que su oferta fue elegida).
4. **A – Acknowledge (ACK):** el servidor confirma oficialmente la asignación; la IP ya se puede usar.

Mensajes adicionales: NACK = el servidor rechaza el Request (IP ya no disponible, inválida o que el cliente no puede usar; el cliente reinicia el proceso). **INFORM** = el cliente que **ya tiene IP** (por ejemplo asignada a mano) pide solo otros parámetros (DNS, gateway, etc.).

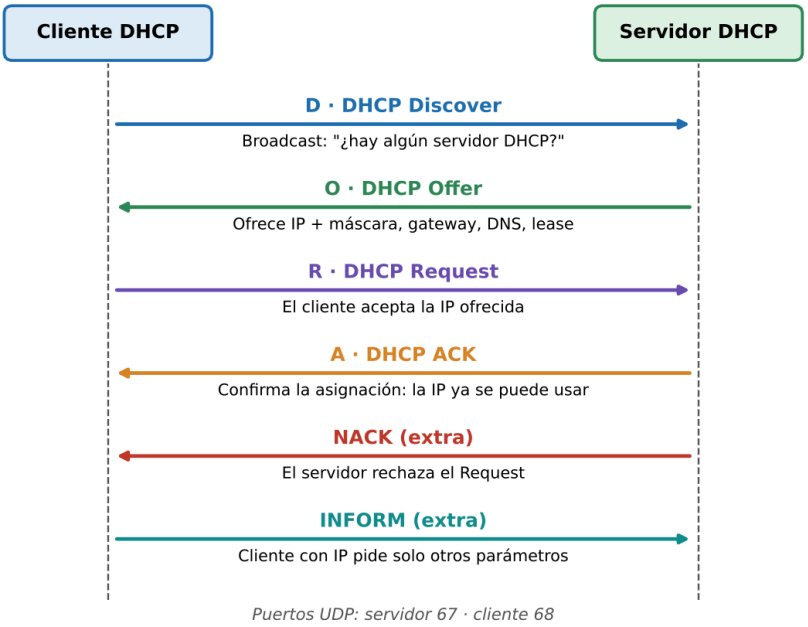


Figura 3. Proceso DORA extendido

4. DNS (Domain Name System)

Traduce **nombres de dominio legibles** (google.com) a **direcciones IP** (142.250.80.46) y viceversa. Es la **"guía telefónica" de Internet**: la red solo entiende direcciones binarias, pero las personas usamos nombres (strings ASCII). Es un conjunto de protocolos y servicios en una red TCP/IP, de uso intensivo en Internet y en empresas (uso interno). La configuración del cliente DNS es parte de los parámetros de red (dada por **DHCP** o configurada a mano).

Historia: por qué existe

- En ARPANET se usaba un archivo **hosts.txt** con todos los hosts y sus IP, que cada host bajaba cada noche. **Enfoque no escalable.**
- Se necesitaba un sistema que evitara conflictos (consistencia de datos, sobrecarga) **sin administración central**.
- Solución: **DNS**, una **base de datos distribuida** con un esquema **jerárquico (árbol)** de nombres.

Jerarquía y dominios

El nombre se lee de izquierda a derecha, pero la jerarquía funciona **al revés**: cada punto separa un nivel y la autoridad va de lo más general a lo más específico (**raíz → TLD → 2.º nivel → subdominios**).

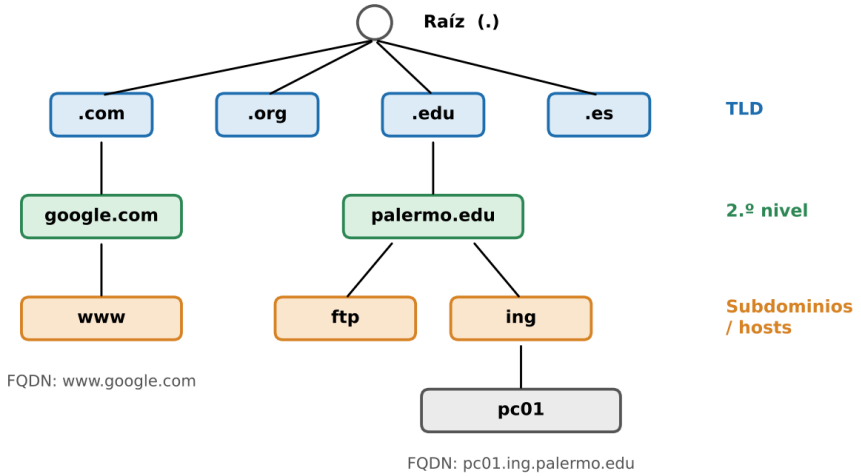


Figura 4. Jerarquía DNS y FQDN

- **Dominio:** cada nodo del árbol junto con todos los nodos debajo de él. Puede contener **hosts** y otros **subdominios**. Ej.: palermo.edu contiene el host ftp.palermo.edu y el subdominio ing.palermo.edu (que contiene pc01.ing.palermo.edu).
- **FQDN (Fully Qualified Domain Name):** nombre completo desde la raíz, ej. www.palermo.edu. Analogía: *full path* vs. ruta relativa en un sistema de archivos.
- **Zona:** porción del espacio de nombres administrada por un servidor, guardada en un **archivo de zona**. Cada zona se ancla en un dominio "raíz de la zona". Un servidor puede manejar uno o varios archivos de zona y **no necesariamente contiene todo el árbol** de subdominios.
- La raíz de la base de datos DNS en Internet la maneja el centro de información de la red (InterNIC).

TLD (Top Level Domains)

Tipo	Ejemplos
gTLD (genéricos)	.COM, .NET, .EDU, .ORG · .GOV (gobierno de EE.UU.) · .MIL (militar de EE.UU.) · .INT (organizaciones internacionales: itu.int, nato.int)
ccTLD (códigos de país)	Códigos de países según ISO-3166 (de dos letras)
TLD adicionales (12)	.AERO, .BIZ, .CAT, .COOP, .INFO, .JOBS, .MOBI, .MUSEUM, .NAME, .PRO, .TEL, .TRAVEL

Tipos de servidores DNS

- **Servidor de nombres:** almacena información del espacio de nombres; es **autoridad** de una o más zonas.
- **Primario:** obtiene los datos de sus **archivos locales**; los cambios (agregar dominios o hosts) se hacen **acá** (modificación centralizada).
- **Secundario (slave):** obtiene los datos de la zona de **otro servidor autoritativo (el primario) por la red**; ese proceso se llama **transferencia de zona**.
- Un mismo servidor puede ser primario para unas zonas y secundario para otras (se define en la configuración de cada zona).
- **Solo caché:** realiza consultas, guarda las respuestas en caché y devuelve resultados; **no es autoritario de ningún dominio**.
- Un servidor DNS puede ser cliente de otro servidor DNS.

¿Para qué sirven los secundarios?

- **Redundancia / tolerancia a fallos:** al menos un primario y un secundario por zona, lo más independientes posible.
- **Ubicaciones remotas:** que los clientes no tengan que cruzar rutas lentas para resolver nombres.
- **Reducir la carga** del servidor primario.

Forwarders (reenviadores)

Resolver nombres **fuera de la zona local** (Internet) requiere hablar con servidores externos. Por **seguridad y disponibilidad** solo servidores específicos (los *forwarders*) hacen la comunicación de área extensa. El resto de los servidores de la empresa se configuran para usar esos reenviadores: si reciben una consulta que no pueden resolver con sus propias zonas, **se la pasan al forwarder**, que hace la resolución y devuelve el resultado al servidor, y este al solicitante original.

Tipos de consulta

Tipo	Cómo es	Quién la hace
Recursiva	El servidor consultado **debe responder** con el dato pedido o con un error (el tipo de consulta o el dominio no existe). Si no puede, pregunta a los servidores "más cercanos conocidos" (ej. para lab.ing.palermo.edu busca primero esa zona, luego ing.palermo.edu, palermo.edu y por último edu).	Un **cliente DNS (resolver)** a un servidor; también un servidor a su forwarder
Iterativa	El servidor consultado da **la mejor respuesta que tiene en ese momento** (por ejemplo, a quién preguntar después).	Un **servidor DNS a otros servidores**, tras recibir una consulta recursiva
Inversa	Obtiene el **nombre a partir de la IP**. Usa el árbol **in-addr.arpa** con los octetos de la IP **en orden invertido** (las IP son más específicas de izq. a der.; los nombres, menos).	Cliente que conoce la IP y busca el nombre

¿Qué pasa cuando escribís www.google.com?

1. **Caché local:** la PC revisa si ya conoce la IP (se puede ver con ipconfig /displaydns).
2. **Resolver:** si no está en caché, el sistema operativo le pregunta al resolver local, intermediario entre el usuario y los servidores DNS.
3. **Servidor recursivo del ISP** (o uno como Google 8.8.8.8 / Cloudflare 1.1.1.1).
4. **Servidor raíz** (uno de los 13 del mundo): "no sé la IP, pero el servidor de .com es este".
5. **Servidor TLD (.com):** "no sé la IP, pero el servidor de google es este".
6. **Servidor autoritativo:** el de Google, que tiene la respuesta real (142.250.80.46).
7. **Respuesta:** la IP vuelve por la cadena hasta el navegador, que se conecta al servidor web.

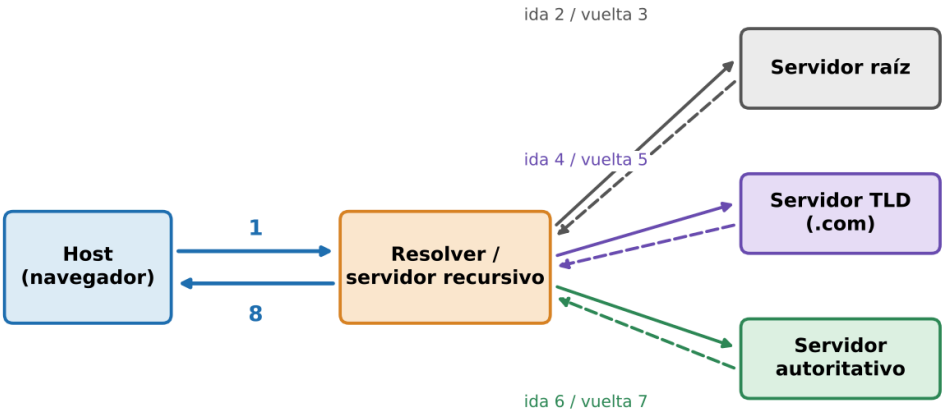


Figura 5. Resolución de un nombre DNS

Registros de recursos (Resource Records)

DNS es una **base de datos distribuida** organizada en registros de recursos. Cada registro tiene **nombre** (dominio al que pertenece), **tipo** (qué clase de información) y **valor** (el dato). El **orden** de los registros no importa y el **tiempo de vida (TTL, en segundos)** indica la estabilidad de la información y controla la expiración de las copias.

Tipo	Función	Ejemplo
A	Nombre de dominio → dirección **IPv4**	google.com → 142.250.80.46
AAAA	Nombre de dominio → dirección **IPv6**	google.com → 2607:f8b0:4004:c09::64
CNAME	**Alias**: un dominio que apunta a otro dominio, no a una IP	www.google.com → google.com
MX	Servidor de **correo** responsable de recibir los emails del dominio	google.com → aspmx.l.google.com

5. Correo electrónico: SMTP, POP e IMAP

Es uno de los servicios más importantes de Internet: permite intercambiar “cartas” de forma **fácil, rápida y barata**, a una multitud de destinatarios, con esquema **cliente-servidor**.

Conceptos y estándares

- **Conceptos:** cuenta de correo (usuario@dominio.com, con usuario y contraseña), buzón, alias y lista de correo.
- **Clientes:** con interfaz gráfica (Outlook, Thunderbird, Apple Mail), de texto (pine, elm, mail) o web (Gmail, Hotmail, Yahoo!).
- **Estándares:** SMTP, IMF (Internet Mail Format), MIME, POP, IMAP.

Componentes

Componente	Función
MUA (Mail User Agent)	**Cliente de correo**: componer, editar y leer mensajes. Usa dos servidores: **saliente (SMTP)** y **entrante (POP o IMAP)**.

Componente	Función
MTA (Mail Transfer Agent)	**Servidor de correo**: almacena los correos de los remitentes para su envío (**cola saliente**) y los correos entrantes de sus usuarios.
MDA (Mail Delivery Agent)	**Agente de envío/entrega**: copia los mensajes entrantes al **buzón** del usuario.

Escenario: Alice le envía un mensaje a Bob

1. Alice usa su **MUA** para componer el mensaje dirigido a bob@dominioBob.es.
2. El MUA de Alice lo envía a **su servidor de correo**; el mensaje queda en la **cola de mensajes salientes**.
3. El **cliente SMTP** del servidor de Alice abre una **conexión TCP** con el servidor de correo de Bob.
4. El cliente SMTP **envía el mensaje** sobre esa conexión TCP.
5. El servidor de Bob **coloca el mensaje en el buzón** de Bob.
6. Bob usa su cliente de correo para acceder a los mensajes entrantes (**POP o IMAP**) y leerlos.

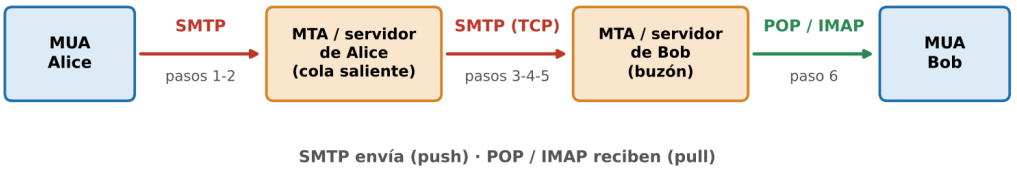


Figura 6. Alice envía un mensaje a Bob

Formato del mensaje

- **IMF (RFC 5322): cabeceras** (To, From, Subject, Date) + **línea en blanco** + **cuerpo**. Texto simple (ASCII, no extendido) de hasta 998 caracteres por línea (sin CRLF).
- **MIME:** agrega funcionalidad: **ficheros adjuntos** y **ASCII extendido**. Nuevas cabeceras: Mime-Version, Content-Type (por defecto text/plain; con adjuntos, Multipart), Content-Description y Content-Transfer-Encoding.
- **Codificaciones:** 7 bits, 8 bits y binary, **quoted-printable** y **base64**. Ej. quoted-printable: ó = =F3 y ñ = =F1 (“Transmisión de ñ” → “Transmisi=F3n de =F1”).

SMTP (RFC 5321)

- Funcionamiento sencillo **cliente-servidor**, usado entre **MUA** → **MTA** y **MTA** → **MTA**. Usa **TCP, puerto 25**.
- **Tres fases:** handshaking (saludo) → transferencia del mensaje (pueden ser varios) → cierre de conexión.
- Los mensajes se codifican en **ASCII de 7 bits** (binario → ASCII al enviar, ASCII → binario al recibir).
- **Respuesta:** texto libre + **código de estado de 3 cifras**; la primera cifra indica éxito o fracaso: **4xx = error temporal**, **5xx = error permanente**.

Comando	Función
HELO	Saludo tras aceptar la conexión

Comando	Función
MAIL FROM	Identifica al remitente
RCPT TO	Indica el destinatario
DATA	Inicio del mensaje; el fin se marca con una línea que contiene solo un punto “.”
QUIT	Cierra la sesión SMTP

POP3 (RFC 1939) e IMAP

Ambos son para **recibir y leer** correo (SMTP solo envía). Diferencia clave:

	POP3	IMAP
Puerto	110	143
Mensajes	Los **descarga** al dispositivo y los **elimina del servidor**	Los **mantiene en el servidor** y los sincroniza entre dispositivos
Uso típico	Acceso desde **un solo equipo**	Celular, PC y webmail a la vez (estándar moderno)
Complejidad	Simple	Más complejo: permite **carpetas en el servidor**, acceso a **partes de un mensaje** y **conserva el estado** entre sesiones. El correo nuevo llega a la carpeta **INBOX**

POP3 tiene dos fases. Autorización: el cliente envía **user** (nombre de cuenta) y **pass** (contraseña); el servidor responde **+OK** o **-ERR**. **Transacción: list** (lista los identificadores de mensajes), **retr** (descarga el mensaje indicado), **dele** (borra el mensaje) y **quit**.

6. HTTP y HTTPS

- **HTTP (HyperText Transfer Protocol):** transmite archivos de hipertexto (HTML y multimedia) por Internet; se usa en **cada transacción de la web**. Puerto **TCP 80**.
- Define la **sintaxis y semántica** que usan clientes, servidores y proxies. Es un protocolo **orientado a transacciones**, con esquema **petición-respuesta**.
- La información transmitida se llama **recurso** y se identifica con una **URL**.
- **HTTPS:** HTTP + una capa de cifrado **TLS**; los datos viajan cifrados y se garantiza que el sitio sea quien dice ser. Puerto **443**. Hoy casi todo el tráfico web lo usa.
- **HTML:** lenguaje de **marcas** para estructurar textos y presentarlos como hipertexto; es el formato estándar de las páginas web.

Arquitectura web cliente/servidor

Elemento	Función
Cliente	Ejecuta un intérprete de HTML (el navegador)
Servidor	Controla el acceso, sirve la página solicitada y puede ejecutar aplicaciones especiales (ej. procesar formularios)
Backend	Entrega datos al servidor
Servidor de aplicaciones	Enlaza el servidor con la base de datos

URL (Uniform Resource Locator)

Secuencia de caracteres con formato estándar que **nombra recursos** (documentos, imágenes) por su **localización**. Es la **dirección** del recurso en Internet y permite que el navegador lo encuentre y lo muestre.

Métodos HTTP

HTTP **no maneja persistencia (es sin estado)**: no almacena información sobre conexiones, envíos ni destinatarios.

Método	Función
GET	Devuelve el recurso identificado en la URL pedida
HEAD	Como GET pero **sin el cuerpo**: solo devuelve la información de cabecera
POST	Prepara al servidor para **recibir información del cliente**; suele usarse para enviar formularios
PUT	Envía el recurso identificado en la URL **del cliente al servidor**
DELETE	Pide al servidor que **borre** el recurso identificado con la URL
OPTIONS	Pide información sobre las características de comunicación del servidor; permite negociar parámetros
TRACE	Inicia un ciclo de mensajes de petición; sirve para **depuración** (ver qué recibe el servidor)
CONNECT	Reservado para **proxys**: permite convertir un proxy dinámicamente en un túnel (ej. comunicaciones con SSL)

Cookies

Para lograr persistencia en HTTP se usan **cookies**: un fragmento de información que se **guarda en el disco del visitante** a través del navegador, **a pedido del servidor**, y que el servidor puede recuperar en visitas posteriores. Usos: **llevar control de usuarios**, **ofrecer opciones de diseño** y **conseguir información sobre hábitos de navegación**. Se pueden administrar con JavaScript.

7. FTP y TFTP: transferencia de archivos

Objetivos: compartir archivos entre equipos remotos, con sistemas de archivos de cliente y servidor **independientes** y transferencia de datos eficaz. Hay dos protocolos principales:

	FTP (RFC 959)	TFTP (RFC 1350)
Transporte	**TCP** (fiable)	**UDP**, puerto **69** (no fiable)
Puertos	20 (datos) y 21 (control)	69
Características	Completo: navegar directorios, subir y bajar archivos	Muy simple, **sin carpetas ni encriptación**, para **archivos pequeños**

FTP: modelo cliente/servidor

- **Cliente:** quien inicia la transferencia (hacia o desde la parte remota). **Servidor:** el equipo remoto.
- Usa **dos conexiones TCP**: **control (puerto 21)**, para moverse por directorios y ordenar subidas/bajadas, y **datos (puerto 20)**, para los datos transferidos.

- El cliente inicia la conexión **al puerto 21** del servidor. En el establecimiento se negocian el **puerto de datos**, el **modo de conexión (activo/pasivo)** y el **modo de transferencia (ASCII/binario)**.
- Limitación: **no cifra** el tráfico (credenciales y archivos viajan en texto plano). Versiones seguras: **SFTP** (sobre SSH) y **FTPS** (FTP con TLS).

	Modo ACTIVO (estándar)	Modo PASIVO
Control	Puerto aleatorio del cliente (>1024) → puerto 21 del servidor	Igual; el cliente envía el comando **PASV**
Datos	Tras el ACK, del **puerto 20 del servidor** al puerto del cliente indicado en el comando de control (**PORT**)	El servidor indica un **puerto aleatorio (>1024)** y el **cliente** establece la conexión de datos
Quién inicia los datos	El **servidor**	El **cliente**

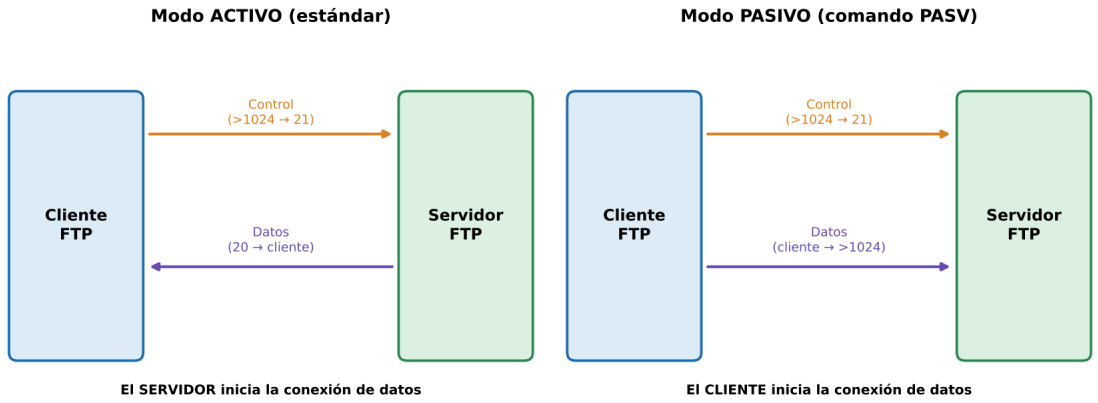


Figura 7. FTP: modo activo vs. modo pasivo

8. SSH (Secure Shell)

Protocolo de capa 7 que permite una **conexión segura** para acceder y administrar servidores de forma **remota** mediante un intérprete de comandos. Dos roles: **cliente SSH** (quien inicia la conexión) y **servidor SSH** (quien la recibe). Además permite **copiar datos de forma segura**, **gestionar claves RSA** y **tunelizar** el tráfico de otras aplicaciones.

Encriptación híbrida: primero asimétrica, luego simétrica

- **Fase 1 – Asimétrica (clave pública/privada):** el servidor presenta su **clave pública**; el cliente la usa para cifrar y enviar una **clave de sesión**. Solo el servidor puede descifrarla con su **clave privada**, así nadie la intercepta. Es **costosa computacionalmente**, por eso no se usa para todo el tráfico.
- **Fase 2 – Simétrica:** ambos extremos comparten la clave de sesión y la usan para **cifrar toda la comunicación posterior**. Es mucho **más rápida y eficiente**.
- La **asimétrica resuelve el intercambio seguro de la clave**; la **simétrica** da el **rendimiento**. Luego se autentica el usuario (contraseña o clave pública).

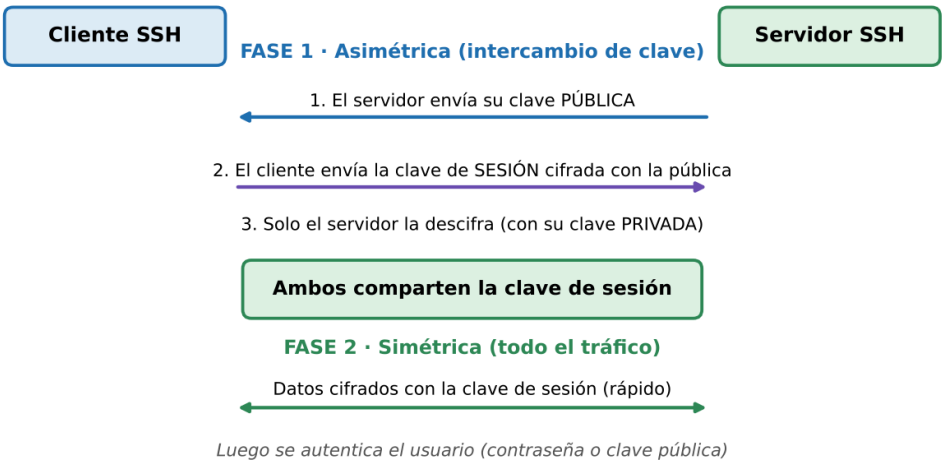


Figura 8. Encriptación híbrida en SSH

9. SMB (Server Message Block)

Permite **compartir archivos, carpetas e impresoras dentro de una red local**. Puerto **445**. Es lo que usa **Windows** para montar unidades de red (\\servidor\carpeta). A diferencia de FTP (que conecta con un servidor remoto), SMB está pensado para **redes internas**. En Linux su implementación se llama **Samba**.

10. Tabla final y claves para el parcial

Protocolo	Para qué sirve	Detalle a recordar
DHCP	Asignar IP y parámetros	DORA; broadcast; lease; NACK / INFORM
DNS	Nombre ↔ IP	Jerarquía raíz → TLD → 2.º nivel; registros A, AAAA, CNAME, MX; recursiva / iterativa / inversa
SMTP	Enviar correo	TCP 25; HELO, MAIL FROM, RCPT TO, DATA, QUIT; 4xx temporal, 5xx permanente
POP3	Recibir correo	110; descarga y borra del servidor; user / pass / list / retr / dele / quit
IMAP	Recibir correo	143; sincroniza, carpetas en el servidor
HTTP / HTTPS	Web	80 / 443; sin estado; GET, POST, PUT, DELETE…; cookies; HTTPS = HTTP + TLS
FTP	Transferir archivos	TCP 20 y 21; activo vs. pasivo; sin cifrado
TFTP	Transferir archivos pequeños	UDP 69; sin carpetas ni cifrado
SSH	Acceso remoto seguro	Cifrado híbrido: asimétrico (clave de sesión) y luego simétrico
SMB	Compartir archivos en LAN	445; Samba en Linux

Para repasar antes del examen

• La capa de aplicación **no es** el programa: es el conjunto de **protocolos y servicios** que usa el programa.

• **DORA** = Discover, Offer, Request, Acknowledge. Los dos primeros mensajes salen en **broadcast**.

• DNS resuelve **de lo general a lo específico**: raíz → TLD → autoritativo. Recursiva = “resolvelo vos”; iterativa = “dame lo mejor que tengas”.

• **SMTP envía** (push); **POP / IMAP reciben** (pull). POP descarga y borra; IMAP sincroniza.

• **FTP** usa 2 conexiones (control 21, datos 20); en **modo pasivo** el cliente inicia la conexión de datos.

• **SSH** combina cifrado **asimétrico** (intercambio de la clave de sesión) y **simétrico** (el tráfico).